

Instructions

- Create a folder with folder name your **roll number** in /home. Upload your .c files to that folder.
- For each question, submit one .c file, named in the format **q01.c**, **q02.c**, etc.
- Provide commented, indented code. Variables should have meaningful names.
- Marks will be awarded on the basis of how *well-written* the code is and how the program performs on closed test cases.

Questions: 6

Marks: 60

Time: 14:00–16:45

1. Secret Pair Detector

Marks: 10

A secret communication system uses pairs of characters as access keys. A pair is considered a Secret Pair if the absolute difference between the ASCII values of the two characters is exactly 10.

Write a C program that reads two characters and determines whether they form a Secret Pair.

Rules:

- Read two characters from the user.
- Find the absolute difference between their ASCII values.
- If the difference is exactly 10:
 - Print "SECRET PAIR FOUND"
 - Generate the Secret Code by concatenating: First character, Second character, ASCII value of the first character
- Otherwise print: "NO SECRET PAIR"

Hint: Remember that a character in C is internally stored as an integer (ASCII).

Input Format

- Line 1: First character
- Line 2: Second character

Output Format

For a Secret Pair:

```
1 Character 1 : <ch1> (<ASCII of ch1>)
2 Character 2 : <ch2> (<ASCII of ch2>)
3
4 SECRET PAIR FOUND
5 Secret Code : <ch1><ch2><ASCII of ch1>
```

Otherwise:

```
1 Character 1 : <ch1> (<ASCII of ch1>)
2 Character 2 : <ch2> (<ASCII of ch2>)
3
4 NO SECRET PAIR
```

Open Test Cases

Test Case 1

Input:

```
A
K
```

Output:

```
Character 1 :  A (65)
Character 2 :  K (75)

SECRET PAIR FOUND
Secret Code :  AK65
```

Test Case 2

Input:

```
b
l
```

Output:

```
Character 1 :  b (98)
Character 2 :  l (108)

SECRET PAIR FOUND
Secret Code :  bl98
```

Test Case 3

Input:

```
A
D
```

Output:

```
Character 1 :  A (65)
Character 2 :  D (68)

NO SECRET PAIR
```

2. Digital Root of a Number

Marks: 10

The *digital root* of a non-negative integer is obtained by repeatedly summing its digits until only a single digit remains. For example, for 9875: $9 + 8 + 7 + 5 = 29$, then $2 + 9 = 11$, then $1 + 1 = 2$. So the digital root of 9875 is 2, and it took **3** rounds of summation.

You must count how many rounds of digit-summing were required to reach the final single digit. If the input number itself already has a single digit (0–9), no summation is required and the count of rounds is 0.

Use a **do-while** loop for the outer summation process, and a **while** loop to extract digits of the current number.

Input Format

- A single integer n (may be large).

Output Format

- If $n < 0$: print "INVALID INPUT".
- Otherwise print "Digital Root = X" on one line.
- On the next line print "Rounds = Y".

Open Test Cases

Test Case 1

*Input:**Output:*

Test Case 2

*Input:**Output:*

Test Case 3

*Input:**Output:*

Test Case 4

*Input:**Output:*

3. Prime Factorization

Marks: 10

Every integer greater than 1 can be written uniquely as a product of prime powers, e.g. $360 = 2^3 \times 3^2 \times 5^1$. You have to find the prime factorization of a given number n and print each distinct prime factor along with its power, one per line, in **increasing order** of the prime.

- If $n \leq 0$, print "INVALID INPUT".
- If $n = 1$, print "1 has no prime factors" (since 1 has no prime factors).

Use a **for** loop to try candidate divisors and a **while** loop to remove all copies of a found factor.

Input Format

- A single integer n .

Output Format

- For each prime factor p with power e , print " p^e " (e.g. 2^3) on its own line, primes in increasing order.

Open Test Cases

Test Case 1

*Input:**Output:*

Test Case 2

*Input:**Output:*

Test Case 3

*Input:**Output:*

Test Case 4

*Input:**Output:*

4. GCD, LCM and Coprime Check

Marks: 10

Given two positive integers a and b , find their Greatest Common Divisor (GCD) using the **Euclidean Algorithm**, and use it to compute their Least Common Multiple (LCM) using the relation:

$$\text{LCM}(a, b) = \frac{a}{\text{GCD}(a, b)} \times b$$

Two numbers are said to be **coprime** if their GCD is exactly 1.

Implement the Euclidean Algorithm using a **while** loop (repeated application of $\text{gcd}(a, b) = \text{gcd}(b, a \bmod b)$ until the remainder becomes 0).

Input Format

- Two integers a and b on a single line, separated by a space.

Output Format

- If either $a \leq 0$ or $b \leq 0$, print "INVALID INPUT".
- Otherwise, print "GCD = X", then "LCM = Y" on the next line.
- On the final line print "Coprime" if $\text{GCD} = 1$, otherwise print "Not Coprime".

Open Test Cases

Test Case 1

Input:

12 18

Output:

GCD = 6
LCM = 36
Not Coprime

Test Case 2

Input:

7 13

Output:

GCD = 1
LCM = 91
Coprime

Test Case 3

Input:

5 5

Output:

GCD = 5
LCM = 5
Not Coprime

Test Case 4

Input:

0 5

Output:

INVALID INPUT

5. Inverted Full Pyramid of Asterisks

Marks: 10

Write a program that reads a height N from the user and prints an inverted full pyramid of asterisks, N rows tall.

Observe the given examples carefully, to design correctly formatted output. Each row of the pyramid should be centered, with spaces on the left as needed.

You must use only **for** loops (no **while**/**do-while**).

Input Format

- A single integer N .

Output Format

- If $N \leq 0$: print "INVALID INPUT".
- Otherwise, print the N -row inverted pyramid exactly as described above (trailing spaces after the last asterisk on a row are not required).

Open Test Cases

Test Case 1

Input:

Output:

```
*****
*****
***
*
```

Test Case 2

Input:

Output:

6. Multi-Distance Calculator

Marks: 10

Given two points $P = (x_1, y_1, z_1)$ and $Q = (x_2, y_2, z_2)$ in three-dimensional space, compute the four distance metrics listed below using the corresponding formulas. A **menu** is presented repeatedly until the user selects option 5 (Quit). Each metric must be implemented as a **separate function**.

The four distance formulas are:

- **Euclidean Distance** (straight-line):

$$d_E = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2}$$

- **Manhattan Distance** (city-block):

$$d_M = |x_2 - x_1| + |y_2 - y_1| + |z_2 - z_1|$$

- **Chebyshev Distance** (chessboard):

$$d_C = \max(|x_2 - x_1|, |y_2 - y_1|, |z_2 - z_1|)$$

- **Minkowski Distance** (general order p , supplied by user):

$$d_p = \left(|x_2 - x_1|^p + |y_2 - y_1|^p + |z_2 - z_1|^p \right)^{1/p} \quad (p \geq 1)$$

When $p = 1$ this equals Manhattan; when $p = 2$ this equals Euclidean.

- If the Minkowski order $p < 1$, print "INVALID ORDER" and re-show the menu.
- Include `<math.h>` and compile with `-lm`. Use `fabs()`, `sqrt()`, `pow()` as needed.

Functions to implement:

- `void printMenu()` — displays the 5-option menu.
- `double euclidean(double x1, double y1, double z1, double x2, double y2, double z2)`
- `double manhattan(double x1, double y1, double z1, double x2, double y2, double z2)`
- `double chebyshev(double x1, double y1, double z1, double x2, double y2, double z2)`
- `double minkowski(double x1, double y1, double z1, double x2, double y2, double z2, double p)`

Input Format

- First line: six space-separated `double` values $x_1 \ y_1 \ z_1 \ x_2 \ y_2 \ z_2$ (coordinates of Point A then Point B).
- Subsequent lines: menu choice (integer 1–5). If choice is 4, the next line contains the Minkowski order p .

Output Format

- For each distance choice, print one line: "<Metric> Distance : <value>" with **4 decimal places**.
- For Minkowski, print: "Minkowski Distance (p=<p>) : <value>" with **4 decimal places**.
- On invalid order ($p < 1$): print "INVALID ORDER".
- On invalid menu choice: print "Invalid choice".
- On quit (choice 5): print "Exiting. Goodbye!".

Open Test Cases

Test Case 1

Input:

```
1.0 2.0 3.0 4.0 6.0 8.0
1
2
3
5
```

Output:

```
Euclidean Distance : 7.0711
Manhattan Distance : 12.0000
Chebyshev Distance : 5.0000
Exiting. Goodbye!
```

Test Case 2

Input:

```
0.0 0.0 0.0 3.0 4.0 0.0
1
5
```

Output:

```
Euclidean Distance : 5.0000
Exiting. Goodbye!
```

Test Case 3

Input:

```
1.0 2.0 3.0 4.0 6.0 8.0
4
3
5
```

Output:

```
Enter Minkowski order p:
Minkowski Distance (p=3.0) : 6.0000
Exiting. Goodbye!
```

Test Case 4

Input:

```
1.0 1.0 1.0 1.0 1.0 1.0
1
5
```

Output:

```
Euclidean Distance : 0.0000
Exiting. Goodbye!
```